

SIEM Detection Engineering & SOC Operations

Week 1 Report

SIEM Architecture, Log Collection and Normalization

PREPARED BY

Ammar Yasser AbdulZaher Ahmed

PROJECT

Mini SOC Lab built on Splunk Enterprise

Table of Contents

1. Executive Summary..... 3

2. Project Scope and Environment.....4

3. SIEM Architecture..... 4

4. Log Source Inventory.....7

5. Implementation..... 8

6. Normalization (CIM-Aligned)..... 16

7. Validation and Attack Scenarios..... 18

8. Challenges and Resolutions..... 21

9. Limitations and Production Recommendations..... 23

10. Conclusion and Next Steps..... 23

Appendix A: Configuration Files..... 24

Appendix B: Validation Queries (SPL)..... 26

Appendix C: Simulated Data Composition..... 27

Week 1 Deliverables Verification..... 28

1. Executive Summary

This report documents **Week 1** of the **SIEM Detection Engineering & SOC Operations** project, completed as part of the Digital Egypt Pioneers Initiative (DEPI). The goal of this phase was to build the foundation of a miniature Security Operations Center (SOC) for a fictional mid-sized organization, **NileTech**: a SIEM architecture, the onboarding of log sources from five telemetry categories, and a normalization layer that lets detection logic be written once and applied across different sources.

Splunk Enterprise was deployed as the SIEM platform. Logs from endpoints, network, identity, cloud and applications were forwarded into six dedicated indexes. Three of the sources are collected live from the lab host (Windows endpoint, identity and network telemetry), while two are simulated with realistic schemas (a web application access log and AWS CloudTrail events), because no production web server or cloud account was available in the lab.

Key results

- **Six dedicated indexes** created: `win`, `identity`, `network`, `app`, `cloud` and `linux` (reserved for expansion).
- **Five telemetry categories onboarded**, with more than 18,000 events indexed in the 24-hour validation window.
- **A common, CIM-aligned field model** (`src`, `user`, `action`) applied across Windows Security, AWS CloudTrail and web access logs.
- **Five attack scenarios** embedded in the data and verified in Splunk: brute force, web reconnaissance and SQL injection, persistence, defense evasion, and data exfiltration.
- **One operational issue** (Splunk denied read access to the Windows Firewall log) was diagnosed from Splunk's internal logs and resolved.

The environment is ready for **Week 2**, in which correlation rules will be written against the scenarios above and tuned for false positives and false negatives.

1.1 Deliverables Checklist

The project booklet defines five deliverables for Week 1 (Project 4: SIEM Detection Engineering, Alert Triage & SOC Operations). The table maps each one to the section of this report that covers it.

Table 1. Week 1 deliverables and where they are covered

Required deliverable	Status	Location in this report
SIEM architecture diagram	Complete	Section 3.3 (Figure 1); layers in Section 3.1; index design in Section 3.2
Data source inventory	Complete	Section 4 (source inventory table)
Log onboarding evidence	Complete	Section 5 (configuration and screenshots) and Section 7 (health check and attack scenarios)
Normalized field mapping	Complete	Section 6 (CIM-aligned mapping table and before/after screenshots)
Retention plan	Documented	Section 5.8 (proposed per-index policy; not enforced in the lab)

2. Project Scope and Environment

2.1 Objectives

The Week 1 requirements were to:

- Design a SIEM architecture for a mid-sized organization.
- Cover five log source categories: endpoints, network, identity, cloud and applications.
- Implement log forwarding and normalization using a SIEM platform (Splunk, ELK or Wazuh).
- Produce a SIEM architecture diagram.

2.2 Organization Profile

NileTech is a fictional organization of roughly 500 employees. It runs Windows endpoints with directory-based identity, a public-facing web application, and workloads in AWS. This profile determines which telemetry a SOC would need to monitor, and each category is represented in the lab by a concrete log source (Section 4).

2.3 Lab Environment

Table 2. Lab environment summary

Component	Details
SIEM platform	Splunk Enterprise 10.2, single instance acting as both indexer and search head
Host	Windows 10 machine hosting both the SIEM and the monitored endpoint
Endpoint telemetry	Windows Security, System and PowerShell event logs; Sysmon with the SwiftOnSecurity community configuration
Network telemetry	Windows Defender Firewall log (<code>pfirewall.log</code>) with allowed and blocked connections logged
Simulated sources	Web server access log and AWS CloudTrail JSON, generated with PowerShell using the real log formats
Receiving port	TCP 9997 enabled for future Universal Forwarders

2.4 Platform Selection

Splunk was selected over ELK and Wazuh for three reasons: its search language (SPL) supports the correlation, sequence and statistical logic required in Week 2; Splunk’s Common Information Model (CIM) provides a standard naming target for normalization; and it is widely used in SOC roles, so the skills built here transfer directly to production environments.

3. SIEM Architecture

The architecture follows a layered pipeline: log sources are collected by Splunk inputs, stored in purpose-specific indexes, passed through a normalization layer, and then consumed by the detection, dashboarding and response workflows planned for Weeks 2 to 4. The complete diagram is shown in Section 3.3 (Figure 1).

3.1 Architecture Layers

Table 3. *Architecture layers and their implementation in the lab*

Layer	Purpose	Lab implementation
Sources	Systems producing security-relevant telemetry	Windows host, Windows Firewall, web access log (simulated), CloudTrail (simulated)
Collection	Reading logs from the source	Splunk WinEventLog inputs and file monitors; port 9997 reserved for Universal Forwarders
Indexing	Storing events by domain	Six indexes separated by telemetry category
Normalization	Common field names across sources	Search-time field aliases and evaluated fields: <code>src</code> , <code>user</code> , <code>action</code>
Analytics	Detection, triage and response	Correlation rules (Week 2), dashboard and triage (Week 3), threat intel and SOAR (Week 4)

3.2 Index Design

Events are separated into indexes by telemetry category rather than stored in a single index. This allows retention and access control to be set per data domain, speeds up searches, and keeps each sourcetype together.

Table 4. *Index design*

Index	Contents	Rationale
win	Windows Security, System, PowerShell and Sysmon events	Endpoint telemetry for host-based detection
identity	Authentication and account-management events routed by EventCode	Isolates identity activity (logons, account creation, group changes)
network	Windows Firewall connection log	Network flow visibility
app	Web application access log	Application-layer attacks such as scanning and SQL injection
cloud	AWS CloudTrail events	Control-plane activity in the cloud account
linux	Reserved	Prepared for a Linux server and Universal Forwarder in later weeks

3.3 Architecture Diagram

Figure 1 shows the end-to-end pipeline of the lab, from log sources to the analytics layer planned for Weeks 2 to 4.

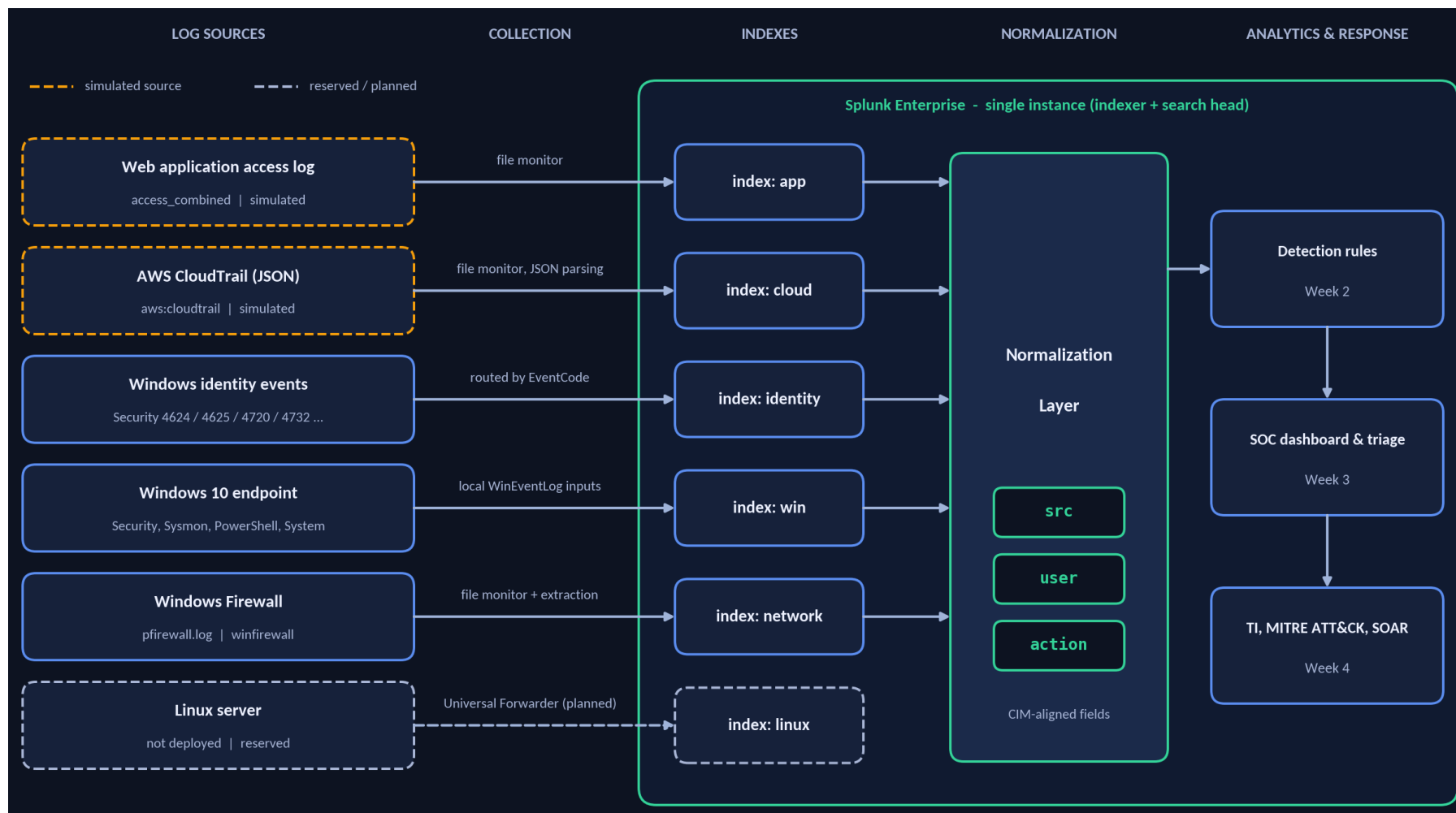


Figure 1. SIEM architecture of the NileTech lab. Dashed orange borders mark simulated sources; dashed grey marks reserved components.

4. Log Source Inventory

The table below lists every source onboarded, how it is collected, and whether the data is live or simulated.

Table 5. Log source inventory

Category	Source	Collection	Index	Sourcetype	Status
Endpoint	Windows Security, System, PowerShell	WinEventLog inputs	win	WinEventLog:*	Live
Endpoint	Sysmon Operational	WinEventLog input (XML)	win	XmlWinEventLog:Microsoft-Windows-Sysmon/Operational	Live
Identity	Windows account and logon events (4624, 4625, 4720, 4726, 4732 and others)	Security log routed by EventCode	identity	WinEventLog:Security	Live
Network	Windows Firewall log	File monitor + field extraction	network	winfirewall	Live
Application	Web server access log	File monitor	app	access_combined	Simulated
Cloud	AWS CloudTrail (JSON)	File monitor, JSON parsing	cloud	aws:cloudtrail	Simulated
Endpoint	Linux server (auditd, auth.log)	Universal Forwarder (planned)	linux	n/a	Reserved

Simulated sources use the **real schema** of the corresponding product (Apache combined log format and the CloudTrail event structure), so detection logic written here carries over to production data. Their composition is described in Appendix C.

5. Implementation

5.1 Splunk Installation

Splunk Enterprise was installed on the lab host and accessed through the web interface on port 8000.

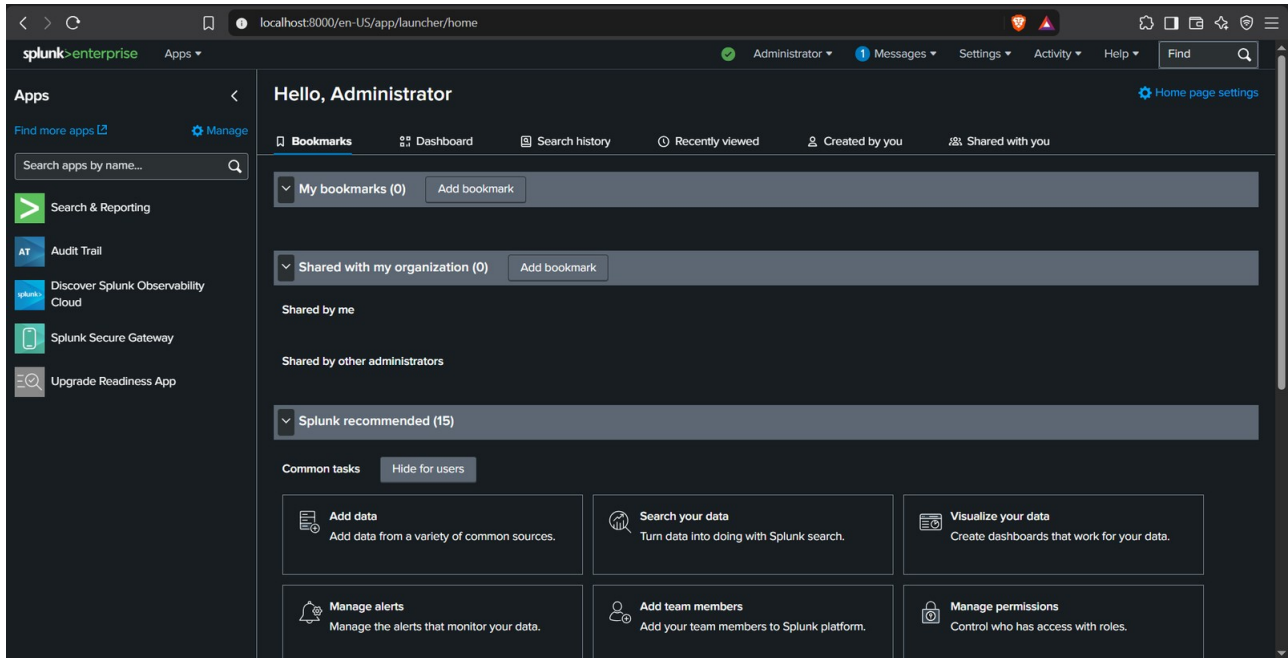


Figure 2. Splunk Enterprise home page after installation.

5.2 Indexes and Receiving Port

Six indexes were created from **Settings > Indexes**, one per telemetry category.

Index Name	Status	Type	Source Type	Size	Other Details
._internal	Active	Events	system	130 MB	488.28 GB, 981K, 20 days ago, a few seconds ago, \$SPLUNK_DB/_internaldb/db
._introspection	Active	Events	system	651 MB	488.28 GB, 163K, 14 days ago, a few seconds ago, \$SPLUNK_DB/_introspection/db
._metrics	Active	Metrics	system	81 MB	488.28 GB, 669K, 14 days ago, a few seconds ago, \$SPLUNK_DB/_metrics/db
._metrics_rollup	Active	Metrics	system	1 MB	488.28 GB, 0, --, --, \$SPLUNK_DB/_metrics_rollup/db
._telemetry	Active	Events	system	2 MB	488.28 GB, 60, 20 days ago, 5 minutes ago, \$SPLUNK_DB/_telemetry/db
._thefishbucket	Active	Events	system	1 MB	488.28 GB, 0, --, --, \$SPLUNK_DB/_thefishbucket/db
app	Active	Events	search	1 MB	500 GB, 220, an hour ago, 25 minutes ago, \$SPLUNK_DB/app/db
cloud	Active	Events	search	1 MB	500 GB, 102, 10 hours ago, an hour ago, \$SPLUNK_DB/cloud/db
history	Active	Events	system	1 MB	488.28 GB, 0, --, --, \$SPLUNK_DB/historydb/db
identity	Active	Events	search	1 MB	500 GB, 20, 25 minutes ago, 25 minutes ago, \$SPLUNK_DB/identitydb/db
linux	Active	Events	search	1 MB	500 GB, 0, --, --, \$SPLUNK_DB/linuxdb/db
main	Active	Events	system	1 MB	488.28 GB, 0, --, --, \$SPLUNK_DB/defaultdb/db
network	Active	Events	search	1 MB	500 GB, 4.82K, an hour ago, a few seconds ago, \$SPLUNK_DB/networkdb/db
project	Active	Events	search	1 MB	500 GB, 0, --, --, \$SPLUNK_DB/projectdb/db
splunklogger	Inactive	Events	system	0 B	488.28 GB, 0, --, --, \$SPLUNK_DB/splunkloggerdb/db
summary	Active	Events	system	1 MB	488.28 GB, 0, --, --, \$SPLUNK_DB/summarydb/db
win	Active	Events	search	12 MB	500 GB, 51.4K, 2 months ago, a few seconds ago, \$SPLUNK_DB/win/db

Figure 3. Indexes page showing the six project indexes (app, cloud, identity, linux, network, win) alongside Splunk's built-in indexes.

The **win** index shows events from two months earlier because the Windows Security input was configured with `start_from = oldest`, which back-fills existing log history on first read.

Receiving port **9997** was also enabled. In this single-host lab, logs are read through local inputs, so the port is not yet receiving data; it is enabled so that Universal Forwarders from additional hosts can be added without changing the indexer.

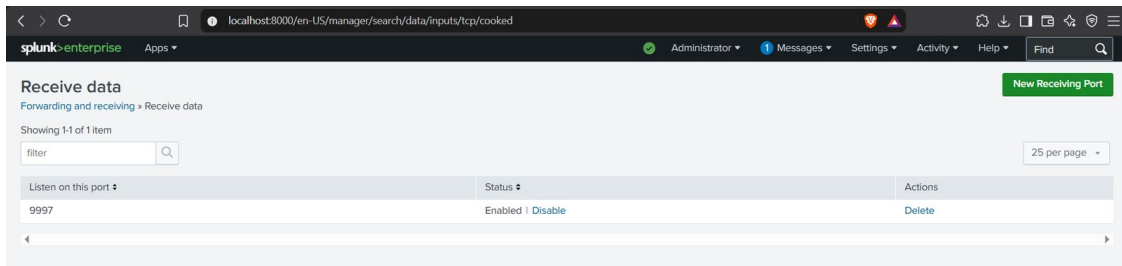


Figure 4. Receiving port 9997 enabled.

5.3 Endpoint Telemetry

Sysmon was installed with the SwiftOnSecurity community configuration to record process creation, network connections and registry changes. Windows audit policy was extended so that logon events, process creation and user-account management are written to the Security log, and command-line capture was enabled for process creation events.

```
auditpol /set /subcategory:"Logon" /success:enable /failure:enable
auditpol /set /subcategory:"Process Creation" /success:enable
auditpol /set /subcategory:"User Account Management" /success:enable /failure:enable
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" ^
/v ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
```

```
TimeCreated : 10/4/2026 5:22:39 PM
Id : 13
Message : Registry value set:
RuleName: Inv08-Ver
EventType: SetValue
UtcTime: 2026-10-04 14:22:39.329
ProcessGuid: {6f218e54-2aca-6ac2-5b00-000000003100}
ProcessId: 4788
Image: C:\Windows\System32\svchost.exe
TargetObject: \REGISTRY\A\{02c18c5d-3780-a6f9-15b4-616bd32b0b7b}\Root\InventoryApplicationFile\sysmon64.e
xe|878d5ea8246959be\BinProductVersion
Details: 15.22.0.0
User: NT AUTHORITY\SYSTEM

TimeCreated : 10/4/2026 5:22:39 PM
Id : 13
Message : Registry value set:
RuleName: Inv08-CompileTimeClaim
EventType: SetValue
UtcTime: 2026-10-04 14:22:39.329
ProcessGuid: {6f218e54-2aca-6ac2-5b00-000000003100}
ProcessId: 4788
Image: C:\Windows\System32\svchost.exe
TargetObject: \REGISTRY\A\{02c18c5d-3780-a6f9-15b4-616bd32b0b7b}\Root\InventoryApplicationFile\sysmon64.e
xe|878d5ea8246959be\LinkDate
Details: 09/09/2026 18:45:14
User: NT AUTHORITY\SYSTEM

PS C:\Sysmon> auditpol /set /subcategory:"Logon" /success:enable /failure:enable
The command was successfully executed.
PS C:\Sysmon> auditpol /set /subcategory:"Process Creation" /success:enable
The command was successfully executed.
PS C:\Sysmon> auditpol /set /subcategory:"User Account Management" /success:enable /failure:enable
The command was successfully executed.
PS C:\Sysmon> reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
The operation completed successfully.
PS C:\Sysmon>
```

Figure 5. Sysmon operational events and successful audit policy configuration.

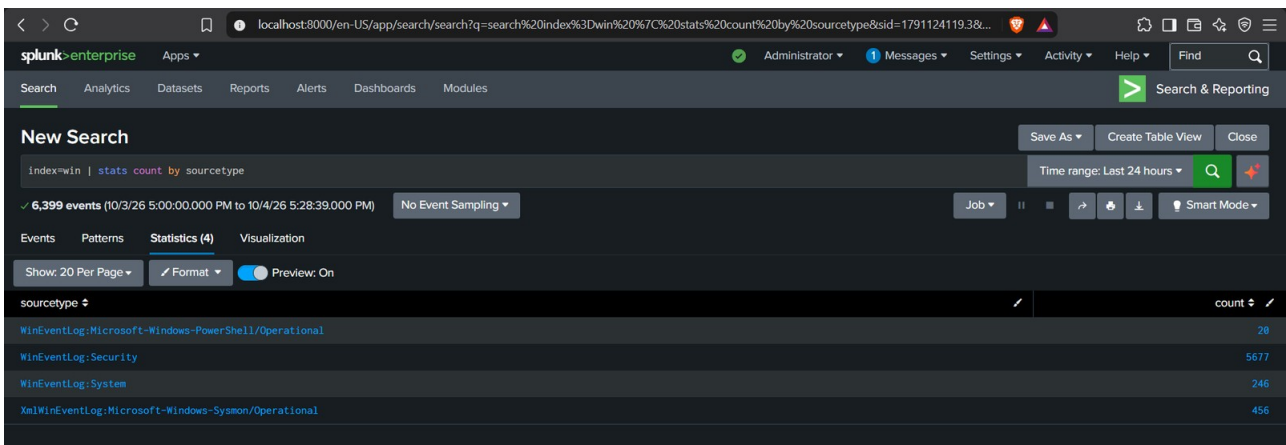
Four Windows event channels were then added to `inputs.conf` and indexed into `win`. The full configuration is in Appendix A.

```

PS C:\Sysmon> auditpol /set /subcategory:"Logon" /success:enable /failure:enable
The command was successfully executed.
PS C:\Sysmon> auditpol /set /subcategory:"Process Creation" /success:enable
The command was successfully executed.
PS C:\Sysmon> auditpol /set /subcategory:"User Account Management" /success:enable /failure:enable
The command was successfully executed.
PS C:\Sysmon> reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
The operation completed successfully.
PS C:\Sysmon> $conf = @'
>> [WinEventLog::Security]
>> index = win
>> disabled = 0
>> start_from = oldest
>> current_only = 0
>> renderXml = false
>>
>> [WinEventLog::System]
>> index = win
>> disabled = 0
>> renderXml = false
>>
>> [WinEventLog::Microsoft-Windows-Sysmon/Operational]
>> index = win
>> disabled = 0
>> renderXml = true
>> sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational
>>
>> [WinEventLog::Microsoft-Windows-PowerShell/Operational]
>> index = win
>> disabled = 0
>> renderXml = false
>> '@
PS C:\Sysmon>
PS C:\Sysmon> Add-Content -Path "C:\Program Files\Splunk\etc\system\local\inputs.conf" -Value $conf
PS C:\Sysmon> Restart-Service splunkd
Restart-Service : Service "splunkd (Splunkd)" cannot be stopped due to the following error: Cannot stop Splunkd service on computer ".".
At line:1 char:1
+ Restart-Service splunkd
+ ~~~~~
+ CategoryInfo          : CloseError: (System.ServiceProcess.ServiceController:ServiceController) [Restart-Service], ServiceCommandException
+ FullyQualifiedErrorId : CouldNotStopService,Microsoft.PowerShell.Commands.RestartServiceCommand
PS C:\Sysmon>

```

Figure 6. Endpoint inputs appended to inputs.conf. The restart error at the bottom was avoided by restarting through the Splunk command line.



The screenshot shows the Splunk Search & Reporting interface. The search bar contains the query `index=win | stats count by sourcetype`. The results show 6,399 events. Below the search bar, there are tabs for Events, Patterns, Statistics (4), and Visualization. The Statistics tab is selected, showing a table with 4 columns: sourcetype, count, and two empty columns. The table lists four sourcetypes with their respective counts.

sourcetype	count
WinEventLog:Microsoft-Windows-PowerShell/Operational	20
WinEventLog:Security	5677
WinEventLog:System	246
XmlWinEventLog:Microsoft-Windows-Sysmon/Operational	456

Figure 7. Validation: four Windows sourcetypes are being indexed into the win index.

To confirm that security-relevant events are captured, eight failed logons were generated with a non-existent account. All eight appear as Event ID 4625 with the failure reason “Unknown user name or bad password”.

_time	Account_Name	Source_Network_Address	Failure_Reason
2026-10-04 17:30:06.950	fakeuser	:::1	Unknown user name or bad password.
2026-10-04 17:30:02.848	fakeuser	:::1	Unknown user name or bad password.
2026-10-04 17:29:58.747	fakeuser	:::1	Unknown user name or bad password.
2026-10-04 17:29:54.606	fakeuser	:::1	Unknown user name or bad password.
2026-10-04 17:29:50.491	fakeuser	:::1	Unknown user name or bad password.
2026-10-04 17:29:46.385	fakeuser	:::1	Unknown user name or bad password.
2026-10-04 17:29:42.258	fakeuser	:::1	Unknown user name or bad password.
2026-10-04 17:29:38.146	fakeuser	:::1	Unknown user name or bad password.

Figure 8. Failed login events (Event ID 4625) from the test account.

5.4 Network Telemetry

The lab has no dedicated firewall appliance, so the built-in Windows Defender Firewall was configured to log both allowed and blocked connections for the Domain, Private and Public profiles. Splunk monitors the resulting `pfirewall.log` file and extracts action, protocol, source and destination fields at search time.

```
Set-NetFirewallProfile -Profile Domain,Private,Public `
-LogAllowed True -LogBlocked True `
-LogFileName "%systemroot%\system32\LogFiles\Firewall\pfirewall.log" `
-LogMaxSizeKilobytes 16384
```

```
PS C:\Sysmon> Invoke-WebRequest https://example.com -UseBasicParsing | Out-Null
PS C:\Sysmon> Get-NetFirewallProfile | Select Name, Enabled, LogAllowed, LogBlocked, LogFileName

Name      : Domain
Enabled   : True
LogAllowed: True
LogBlocked: True
LogFileName : %systemroot%\system32\LogFiles\Firewall\pfirewall.log

Name      : Private
Enabled   : True
LogAllowed: True
LogBlocked: True
LogFileName : %systemroot%\system32\LogFiles\Firewall\pfirewall.log

Name      : Public
Enabled   : True
LogAllowed: True
LogBlocked: True
LogFileName : %systemroot%\system32\LogFiles\Firewall\pfirewall.log

PS C:\Sysmon> Test-Path "C:\Windows\System32\LogFiles\Firewall\pfirewall.log"
True
PS C:\Sysmon> Get-Content "C:\Windows\System32\LogFiles\Firewall\pfirewall.log" -Tail 5
2026-10-04 17:35:55 ALLOW UDP 192.168.1.12 224.0.0.252 49434 5355 0 - - - - - SEND
2026-10-04 17:35:55 ALLOW UDP 192.168.1.12 192.168.1.1 61424 53 0 - - - - - SEND
2026-10-04 17:35:55 ALLOW UDP 192.168.1.12 192.168.238.254 137 137 0 - - - - - SEND
2026-10-04 17:35:55 ALLOW UDP fe80::d70:ac69:cb2c:e3ac ff02::1:3 62825 5355 0 - - - - - SEND
2026-10-04 17:35:55 ALLOW UDP 192.168.1.12 224.0.0.252 62825 5355 0 - - - - - SEND
PS C:\Sysmon>
```

Figure 9. Firewall logging enabled on all profiles, and the most recent log entries.

Ingestion initially failed because Splunk lacked permission to read this file. The diagnosis and fix are described in Section 8. After the fix, the network index received more than 1,900 events within minutes.

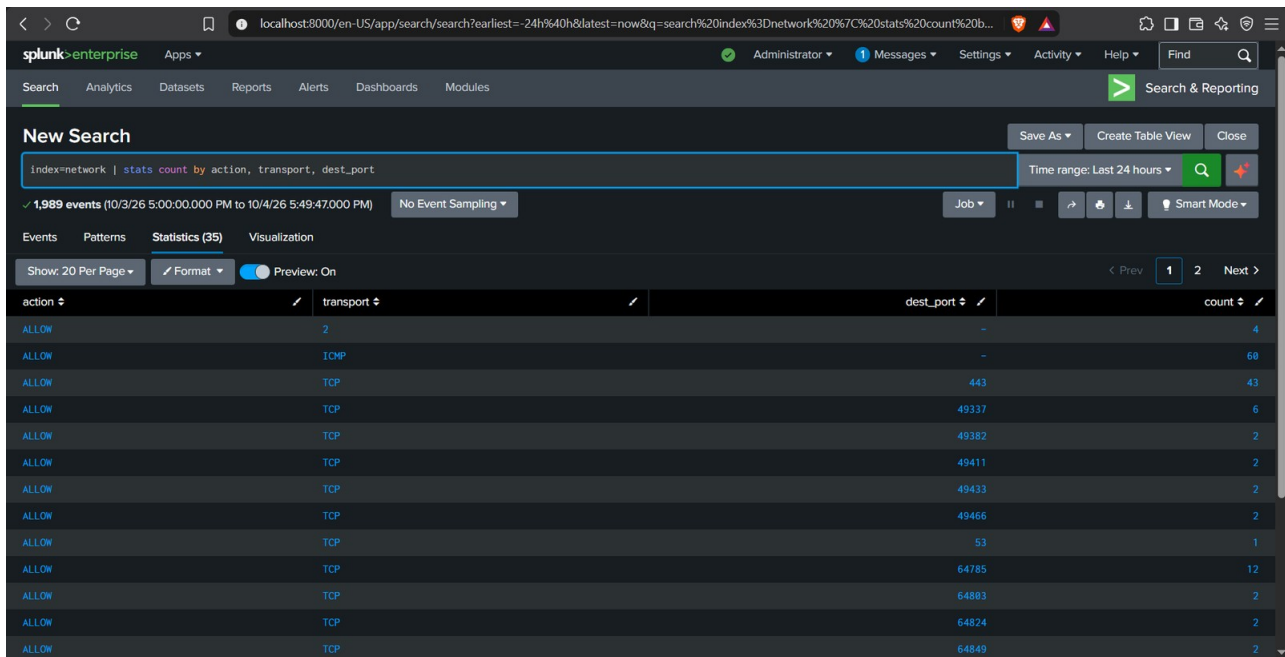


Figure 10. Validation: firewall events in the network index by action, protocol and destination port.

5.5 Identity Telemetry

Identity-related Security events are routed into the dedicated `identity` index at index time. A transform inspects the EventCode of each incoming Security event and redirects those that relate to logons, Kerberos activity and account or group changes. All other Security events remain in `win`.

```
# props.conf
[WinEventLog:Security]
TRANSFORMS-route_identity = route_identity

# transforms.conf
[route_identity]
REGEX      = EventCode=(4624|4625|4634|4648|4672|4720|4722|4724|4726|4728|4732|4738|4740|4768|
4769|4771|4776)
DEST_KEY   = _MetaData:Index
FORMAT     = identity
```

To generate realistic identity activity, a temporary local account was created, added to the Administrators group, subjected to failed logons, and then deleted.

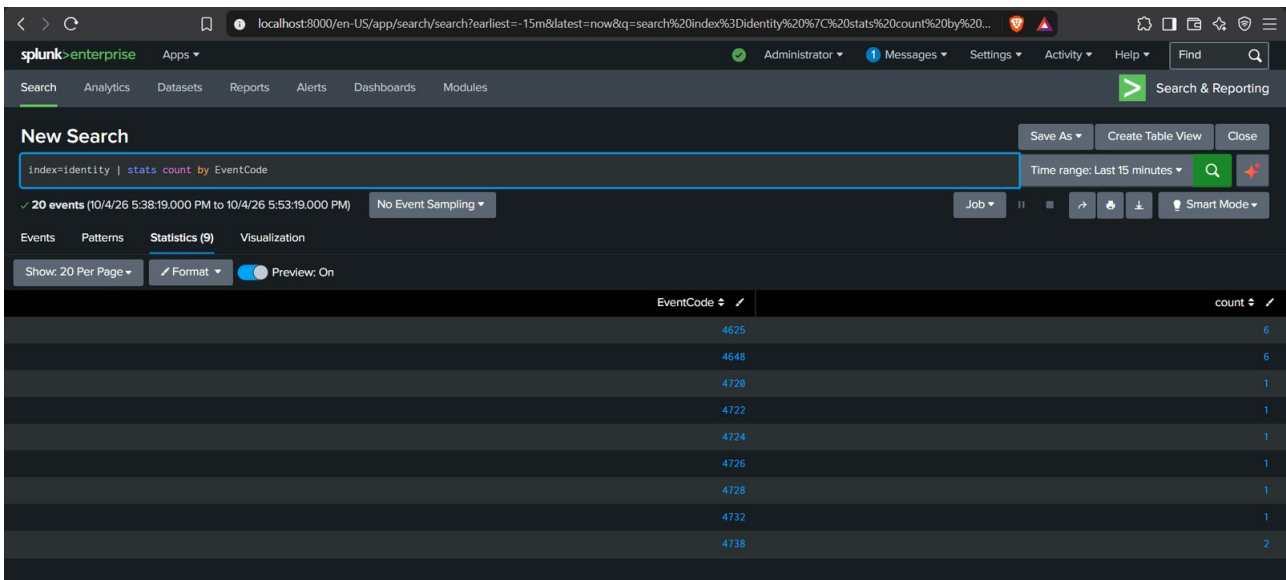


Figure 11. Validation: identity events grouped by EventCode.

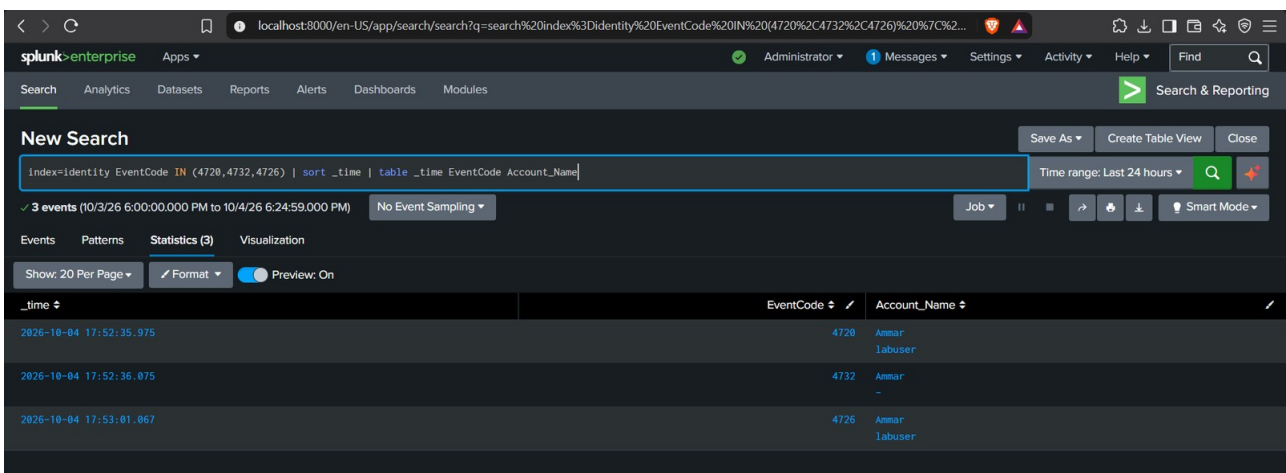


Figure 12. Account lifecycle: creation (4720), group membership change (4732) and deletion (4726) within 30 seconds.

The group-membership event (4732) shows - in one `Account_Name` value because Windows records the added member in a different field from the acting account. This is expected behavior.

5.6 Application Telemetry (Simulated)

A web server access log in the standard combined format was generated with benign traffic from internal clients plus three attacker behaviors: directory scanning, SQL injection attempts and a brute-force attack on `/login`. Splunk reads the file with the built-in `access_combined` sourcetype, which extracts client IP, method, URI and status automatically.

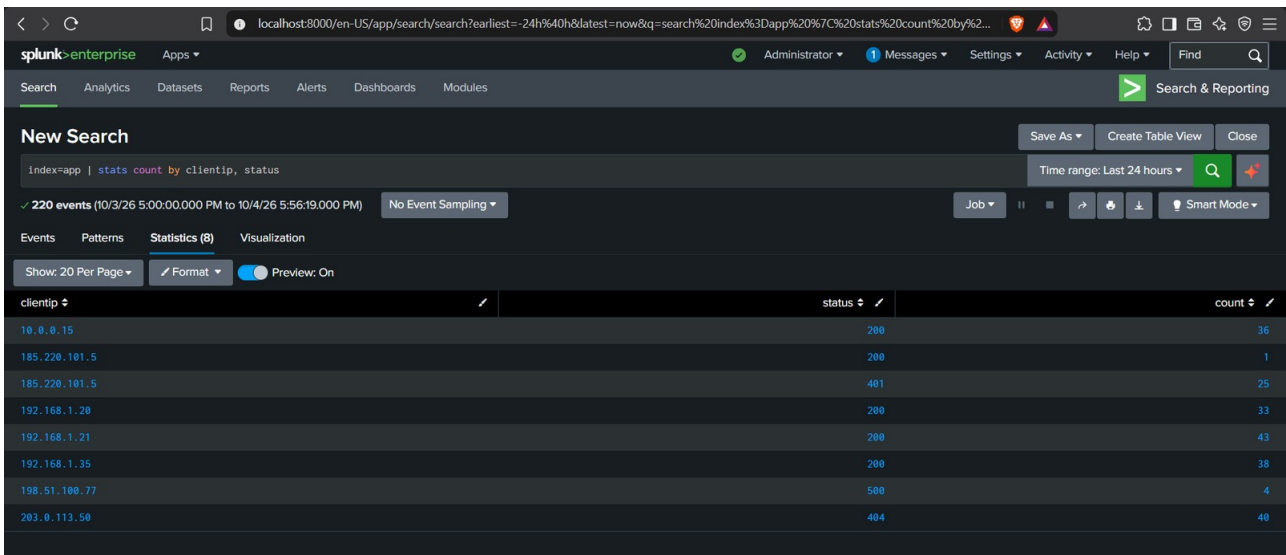


Figure 13. Validation: web requests by client IP and HTTP status.

5.7 Cloud Telemetry (Simulated)

CloudTrail-style JSON events were generated with the real event structure (`eventSource`, `eventName`, `userIdentity`, `sourceIPAddress`, and so on). The sourcetype `aws:cloudtrail` is configured with JSON field extraction and a UTC timestamp format. In production, the same sourcetype would be fed by the Splunk Add-on for AWS.

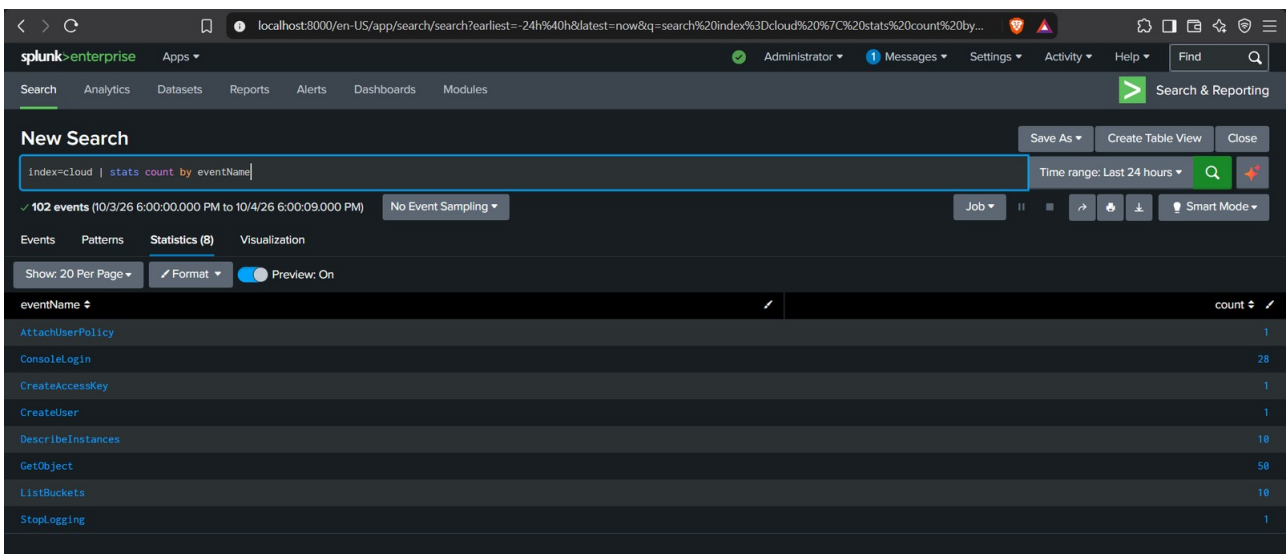


Figure 14. Validation: CloudTrail events by event name.

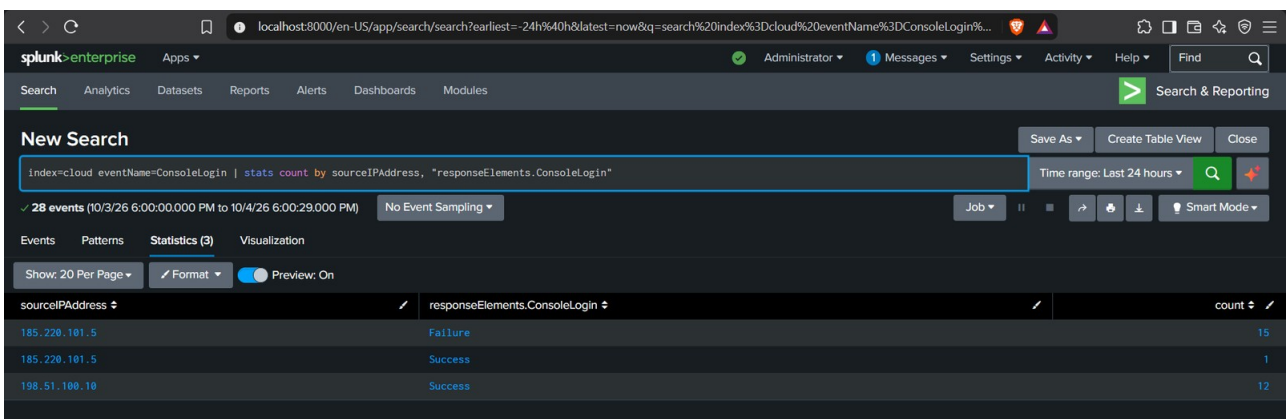


Figure 15. Validation: console logins by source IP and result, showing a failed-login burst followed by a success.

5.8 Retention Plan (Proposed)

Status: proposed, not enforced. The retention periods below are a design for NileTech. They have **not** been applied to the Splunk lab, which runs with default retention, and the `indexes.conf` snippet has not been tested here. It is provided as the configuration to use when the policy is adopted.

Retention is set per index, based on how the data is used and how much of it a medium-sized organization produces. High-volume, short-value data (network flows) is kept for a shorter time; low-volume, high-value data (identity and cloud audit events) is kept longer, because investigations often start weeks after the first malicious action.

Table 6. *Proposed retention policy for NileTech*

Index	Searchable	Archive	Rationale
identity	365 days	+ 2 years	Low volume, high investigative value; account abuse is often discovered late
cloud	365 days	+ 2 years	Control-plane audit trail; commonly required for audit and compliance reviews
win	90 days	+ 9 months	High volume (Security and Sysmon); covers the typical window for intrusion dwell-time investigations
app	90 days	+ 9 months	Web attack patterns are usually found within weeks; archive supports retrospective hunting
linux	90 days	+ 9 months	Same profile as Windows endpoint data
network	30 days	+ 5 months	Highest volume and lowest per-event value; summaries can be kept longer if needed

In Splunk, the searchable period is controlled by `frozenTimePeriodInSecs` in `indexes.conf`; events older than that are moved to the archive location (or deleted if none is set). The values below correspond to the table above.

```
# indexes.conf (retention in seconds)
[identity]
frozenTimePeriodInSecs = 31536000    # 365 days
[cloud]
frozenTimePeriodInSecs = 31536000    # 365 days
[win]
frozenTimePeriodInSecs = 7776000     # 90 days
[app]
frozenTimePeriodInSecs = 7776000     # 90 days
[linux]
frozenTimePeriodInSecs = 7776000     # 90 days
[network]
frozenTimePeriodInSecs = 2592000     # 30 days
```

These values are a design proposal. Before enforcing them, they should be tuned after measuring daily ingest per index and the available storage, and an archive (frozen) location should be configured so that expired events are archived instead of deleted.

6. Normalization (CIM-Aligned)

Each source names the same concept differently: the originating IP is `Source_Network_Address` in Windows, `sourceIPAddress` in CloudTrail and `clientip` in web logs. Without a common model, every detection would need one variant per source.

To solve this, field aliases and evaluated fields were defined in `props.conf` so that all sources expose the same three fields, aligned with the naming used by Splunk's Common Information Model (CIM). They are applied at search time, so no data is re-indexed.

Scope of the normalization. This is **CIM-aligned normalization** (consistent field naming), not full CIM compliance. The Splunk CIM add-on and its data models (for example Authentication and Network Traffic) were **not installed**, so events are not mapped into data models and no data model acceleration is used.

Table 7. Normalization mapping

Source	Original field	Normalized	Method
Windows Security	Source_Network_Address	src	Field alias
Windows Security	Account_Name	user	Last value of the multivalue field, which is the target account
AWS CloudTrail	sourceIPAddress	src	Field alias
AWS CloudTrail	userIdentity.userName	user	Field alias
AWS CloudTrail	responseElements.ConsoleLogin, errorMessage	action	Evaluated: success or failure
Web access log	clientip	src	Field alias
Web access log	status	action	Evaluated: 2xx/3xx = success, 4xx/5xx = failure
Windows Firewall	action (ALLOW / DROP)	action	Evaluated: allowed or blocked; <code>dest</code> also aliased to <code>dest_ip</code>

The following figures show original and normalized fields side by side for each source.

The screenshot shows a Splunk search interface with the following search query: `index=win sourcetype="WinEventLog:Security" EventCode=4625 | head 3 | table _time Source_Network_Address Account_Name src user`. The results table displays the following data:

_time	Source_Network_Address	Account_Name	src	user
2026-10-04 17:30:06.950	::1	- fakeuser	::1	fakeuser
2026-10-04 17:30:02.848	::1	- fakeuser	::1	fakeuser
2026-10-04 17:29:58.747	::1	- fakeuser	::1	fakeuser

Figure 16. Windows: `Source_Network_Address` and `Account_Name` alongside the normalized `src` and `user`.

New Search

index=cloud eventName=ConsoleLogin | head 3 | table _time sourceIPAddress "userIdentity.userName" "responseElements.ConsoleLogin" src user action

Time range: Last 24 hours

3 events (10/3/26 6:00:00.000 PM to 10/4/26 6:23:23.000 PM) No Event Sampling

Events Patterns Statistics (3) Visualization

Show: 20 Per Page Format Preview: On

_time	sourceIPAddress	userIdentity.userName	responseElements.ConsoleLogin	src	user	action
2026-10-04 17:14:45	185.228.101.5	admin	Success	185.228.101.5	admin	success
2026-10-04 17:13:45	185.228.101.5	admin	Failure	185.228.101.5	admin	failure
2026-10-04 17:13:41	185.228.101.5	admin	Failure	185.228.101.5	admin	failure

Figure 17. CloudTrail: original fields alongside normalized src, user and action.

New Search

index=app uri_path="/login" | head 3 | table _time clientip status src action

Time range: Last 24 hours

3 events (10/3/26 6:00:00.000 PM to 10/4/26 6:24:12.000 PM) No Event Sampling

Events Patterns Statistics (3) Visualization

Show: 20 Per Page Format Preview: On

_time	clientip	status	src	action
2026-10-04 17:50:18	185.228.101.5	200	185.228.101.5	success
2026-10-04 17:50:08	185.228.101.5	401	185.228.101.5	failure
2026-10-04 17:50:06	185.228.101.5	401	185.228.101.5	failure

Figure 18. Web log: clientip and status alongside normalized src and action.

The practical benefit is shown by a single query that finds failed logins across three unrelated sources and reports them with identical field names.

New Search

(index=win sourcetype="WinEventLog:Security" EventCode=4625) OR (index=cloud eventName=ConsoleLogin action=failure) OR (index=app uri_path="/login" action=failure)
| eval source_type=case(index="win","Windows", index="cloud","AWS CloudTrail", index="app","Web App")
| stats count by source_type, src, user

Time range: Last 24 hours

48 events (10/3/26 6:00:00.000 PM to 10/4/26 6:03:27.000 PM) No Event Sampling

Events Patterns Statistics (3) Visualization

Show: 20 Per Page Format Preview: On

source_type	src	user	count
AWS CloudTrail	185.228.101.5	admin	15
Web App	185.228.101.5	-	25
Windows	::1	fakeuser	8

Figure 19. One query returning failed authentication from Windows, AWS CloudTrail and the web application with common fields.

The web access log does not record a username, so user is empty for that source. This is a limitation of the log format, not of the normalization.

7. Validation and Attack Scenarios

7.1 Source Health Check

A health check over all indexes confirms that every source is delivering data. The counts below are for the 24-hour window at validation time.

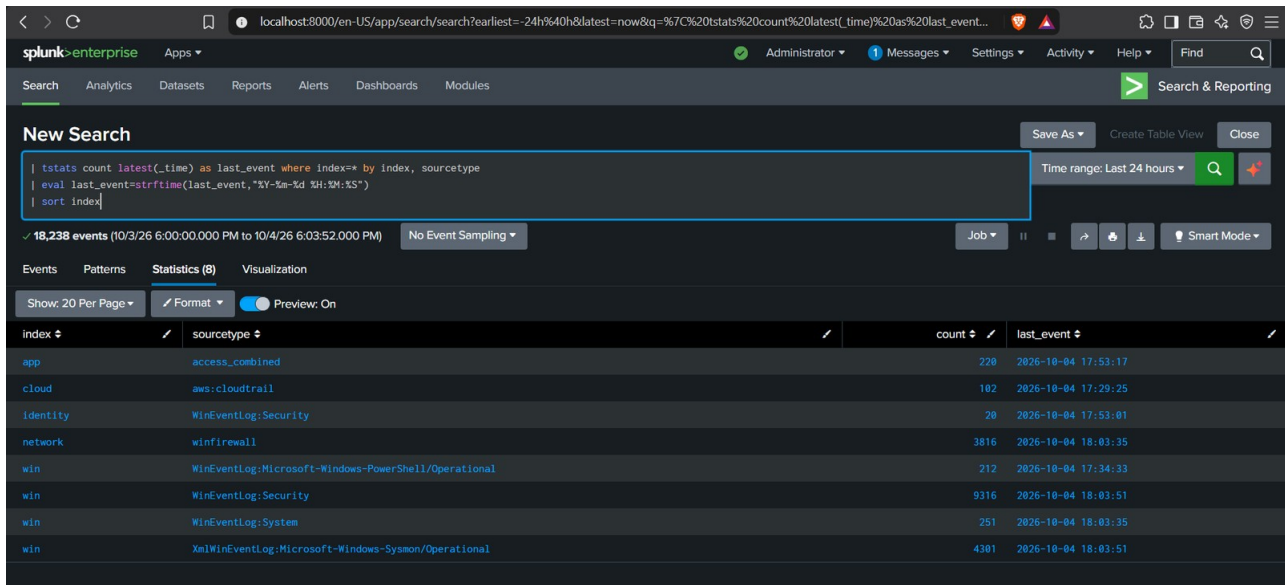


Figure 20. Health check: event count per index and sourcetype.

Table 8. Events per source at validation time

Index	Sourcetype	Events	State
app	access_combined	220	Receiving
cloud	aws:cloudtrail	102	Receiving
identity	WinEventLog:Security	20	Receiving
network	winfirewall	3,816	Receiving
win	WinEventLog:Security	9,316	Receiving
win	XmlWinEventLog:Microsoft-Windows-Sysmon/Operational	4,301	Receiving
win	WinEventLog:System	251	Receiving
win	WinEventLog:Microsoft-Windows-PowerShell/Operational	212	Receiving

7.2 Embedded Attack Scenarios

Five attack scenarios were embedded in the data so that Week 2 detections have known positives to find and benign traffic to avoid. The ATT&CK technique IDs below are a preliminary mapping; they are developed fully in Week 4.

Table 9. Attack scenarios present in the data

Scenario	Evidence in Splunk	Sources	Preliminary ATT&CK
Brute force	Windows: 8 failed logons (4625) for one account. Web: 25 HTTP 401 on /login, then one 200, from one IP. Cloud: 15 failed console logins, then a success, for admin from the same IP.	win, app, cloud	T1110 Brute Force; T1078 Valid Accounts
Web reconnaissance and SQL injection	40 requests returning 404 for admin and config paths from one IP using a scanner user agent; 4 requests with injection payloads returning 500 from a second IP.	app	T1595 Active Scanning; T1190 Exploit Public-Facing Application
Persistence	Cloud: CreateUser , AttachUserPolicy (AdministratorAccess) and CreateAccessKey in sequence. Windows: account created (4720) and added to a group (4732).	cloud, identity	T1136 Create Account; T1098 Account Manipulation
Defense evasion	Cloud: StopLogging on the management trail by the newly created account.	cloud	T1562.008 Disable Cloud Logs
Data exfiltration	50 GetObject calls on the HR data bucket by the new account in 98 seconds from a new IP.	cloud	T1530 Data from Cloud Storage

The persistence and defense-evasion events form a clear sequence in time, with the actor changing from **admin** to the newly created **backdoor-svc** account:

_time	eventName	userIdentity.userName	sourceIPAddress
2026-10-04 17:17:45	CreateUser	admin	185.228.101.5
2026-10-04 17:18:45	AttachUserPolicy	admin	185.228.101.5
2026-10-04 17:19:45	CreateAccessKey	admin	185.228.101.5
2026-10-04 17:20:45	StopLogging	backdoor-svc	185.228.101.5

Figure 21. Cloud attack chain in chronological order.

The exfiltration activity is visible as a single aggregated row, showing volume, duration, actor and source:

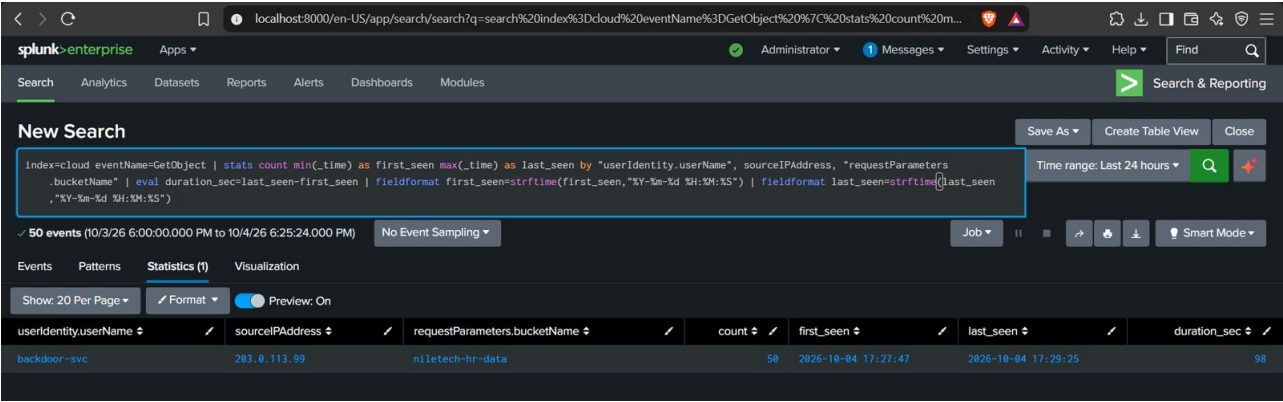


Figure 22. Bulk object download from the HR bucket: 50 GetObject calls in 98 seconds.

8. Challenges and Resolutions

8.1 Splunk Denied Access to the Windows Firewall Log

Symptom. After configuring the firewall log monitor, searches on the `network` index returned no results, although the log file existed and was being written.

Diagnosis. Splunk's own internal logs were searched for the `TailReader` component, which reads monitored files. They contained a warning that Splunk had insufficient permissions to read `pfirewall.log`, with the hint "Access is denied". The warning recurred after a service restart, confirming that the problem was permissions and not configuration.

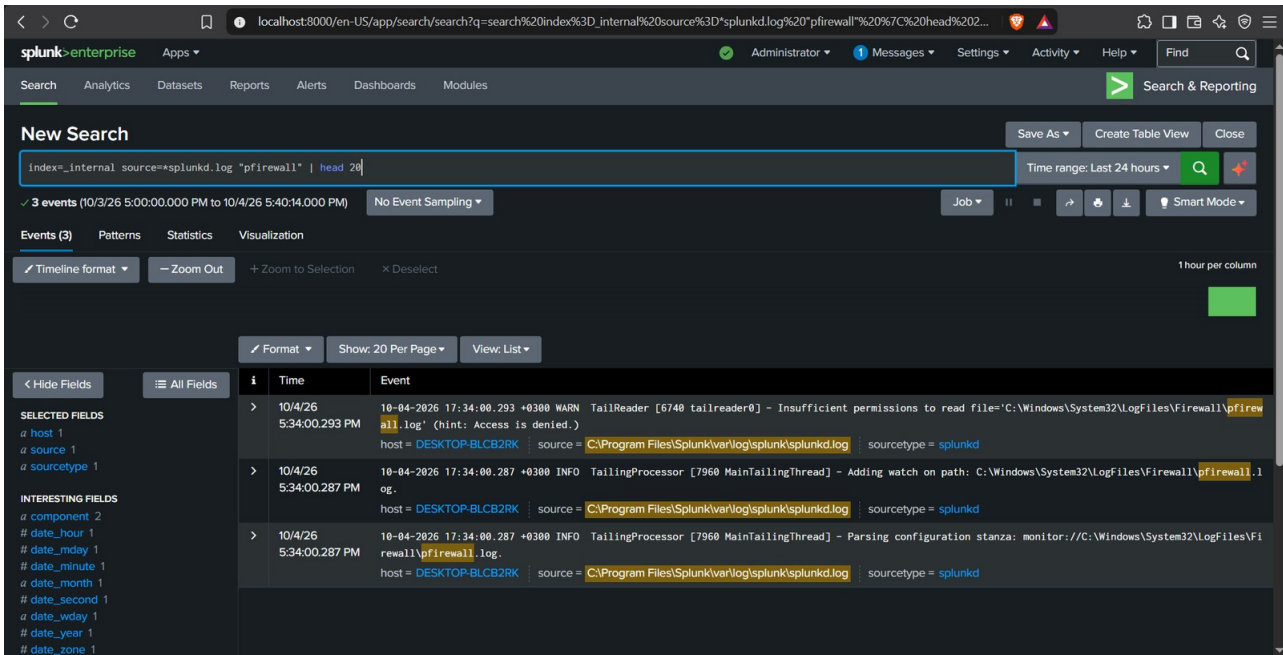


Figure 23. Internal log showing Splunk parsing the monitor stanza and then being denied access.

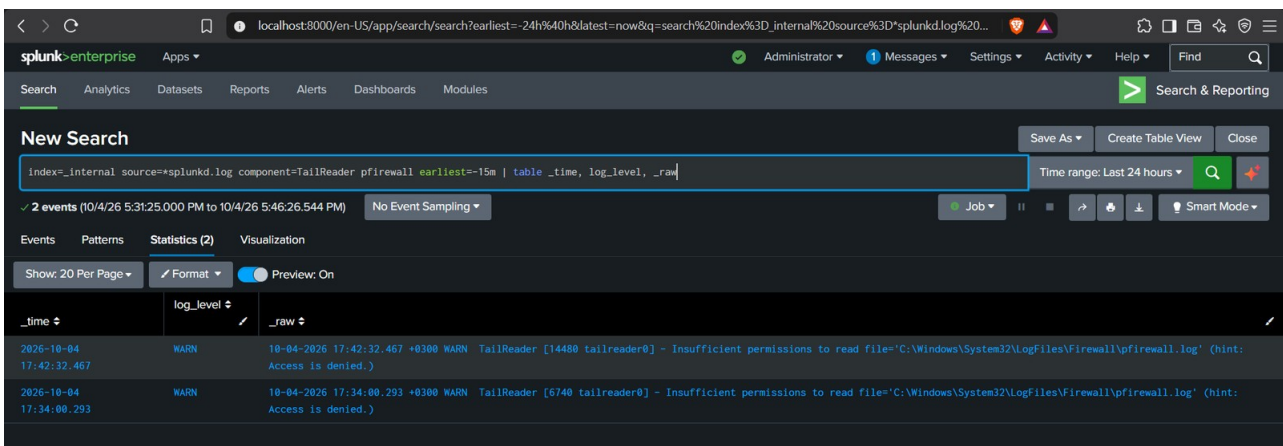


Figure 24. The same access-denied warning recurring after a service restart.

Resolution. The Splunk service (`splunkd`) was reconfigured to run as the `LocalSystem` account, which can read the file. After restarting Splunk, events appeared in the `network` index (Figure 10).

```
sc.exe config splunkd obj= LocalSystem
& "C:\\Program Files\\Splunk\\bin\\splunk.exe" restart
```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Ammar> sc.exe qc splunkd
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: splunkd
        TYPE               : 10  WIN32_OWN_PROCESS
        START_TYPE           : 2    AUTO_START
        ERROR_CONTROL        : 1    NORMAL
        BINARY_PATH_NAME     : "C:\Program Files\Splunk\bin\splunkd.exe" service
        LOAD_ORDER_GROUP     :
        TAG                  : 0
        DISPLAY_NAME         : Splunkd
        DEPENDENCIES         :
        SERVICE_START_NAME   : LocalSystem
PS C:\Users\Ammar>

```

Figure 25. Service configuration confirming splunkd runs as LocalSystem.

Lesson. Troubleshooting started from the data pipeline (source file, then Splunk configuration, then Splunk's internal logs) rather than guessing. In production, a Universal Forwarder with a dedicated least-privilege account and an explicit file ACL is preferable to LocalSystem (see Section 9).

8.2 Other Issues

Table 10. Other issues encountered

Issue	Cause	Resolution
Service restart from PowerShell failed	The Splunk service could not be stopped through the standard service command	Restarted through the Splunk CLI (<code>splunk.exe restart</code>) or the web interface
Network search empty immediately after fix	Search window of 15 minutes did not include back-filled events	Widened the time range to 24 hours
No Domain Controller or cloud account	Lab limited to a single host	Used local Windows identity events and simulated CloudTrail with the real schema
Multivalue Account_Name in Windows events	Events contain both acting and target accounts	Normalized <code>user</code> to the last value with <code>mvindex</code>

9. Limitations and Production Recommendations

The lab is intentionally small. The table below lists where it differs from a production SOC and what would change.

Table 11. *Lab limitations and production recommendations*

Area	Lab	Production recommendation
Collection	Local inputs on the SIEM host	Universal Forwarders on every endpoint, with TLS between forwarders and indexers
Service account	Splunk runs as LocalSystem	Dedicated least-privilege service account with explicit read ACLs on log files
Identity	Local Windows events	Domain Controller logs and Entra ID sign-in logs
Cloud	Simulated CloudTrail	Splunk Add-on for AWS reading CloudTrail from S3/SQS
Network	Windows Firewall log	Perimeter firewall syslog through a syslog collector, plus an IDS such as Suricata
Applications	Simulated access log	Forwarders on web servers, or HTTP Event Collector for application logs
Normalization	Manually defined aliases with CIM-aligned field names	Splunk CIM add-on and data models, plus vendor add-ons, with data model acceleration
Retention and sizing	Splunk defaults; the proposed policy in Section 5.8 is not enforced	Enforce the per-index retention policy, then size license and storage from measured daily ingest
Availability	Single instance	Indexer and search head clustering for resilience
Time	Host clock	Centralized NTP so cross-source correlation is accurate

10. Conclusion and Next Steps

Week 1 delivered a working SIEM foundation: a documented architecture, five telemetry categories flowing into six purpose-built indexes, a common field model across heterogeneous sources, and a dataset that contains known attacks alongside benign traffic. A real ingestion fault was diagnosed from internal logs and resolved, and the production implications were documented.

Plan for Week 2: Detection Rules

- **Brute force** using threshold logic (N failures from one source within a time window) across Windows, web and cloud logs.
- **Suspicious login** using sequence logic (several failures followed by a success).
- **Persistence** using sequence logic (`CreateUser`, `AttachUserPolicy`, `CreateAccessKey` by the same actor).
- **Data exfiltration** using behavioral logic (bulk object downloads compared with a baseline).
- **Lateral movement** and **web attacks**, using the Sysmon, firewall and access-log data already collected.
- **Tuning**: using the benign traffic in the dataset to measure and reduce false positives and false negatives.

Appendix A: Configuration Files

All files are located in C:\Program Files\Splunk\etc\system\local\.

inputs.conf

```
[WinEventLog://Security]
index = win
disabled = 0
start_from = oldest
current_only = 0
renderXml = false

[WinEventLog://System]
index = win
disabled = 0
renderXml = false

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = win
disabled = 0
renderXml = true
sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational

[WinEventLog://Microsoft-Windows-PowerShell/Operational]
index = win
disabled = 0
renderXml = false

[monitor://C:\Windows\System32\LogFiles\Firewall\pfirewall.log]
index = network
sourcetype = winfirewall
disabled = 0

[monitor://C:\SOCLab\logs\webapp_access.log]
index = app
sourcetype = access_combined
disabled = 0

[monitor://C:\SOCLab\logs\cloudtrail_sim.json]
index = cloud
sourcetype = aws:cloudtrail
disabled = 0
```

props.conf

```
[winfirewall]
SHOULD_LINEMERGE = false
TIME_PREFIX = ^
TIME_FORMAT = %Y-%m-%d %H:%M:%S
MAX_TIMESTAMP_LOOKAHEAD = 19
EXTRACT-fw = ^\S+\S+\S+\S+(?<action>\S+)\S+(?<transport>\S+)\S+(?<src>\S+)\S+(?<dest>\S+)\S+(?<src_port>\S+)\S+(?<dest_port>\S+)
FIELDALIAS-norm_dest_ip = dest AS dest_ip
EVAL-action = case(action="ALLOW","allowed", action="DROP","blocked", true(),action)

[aws:cloudtrail]
```

```

SHOULD_LINEMERGE = false
KV_MODE = json
TIME_PREFIX = "eventTime":
TIME_FORMAT = %Y-%m-%dT%H:%M:%SZ
TZ = UTC
FIELDALIAS-norm_src = sourceIPAddress AS src
FIELDALIAS-norm_user = userIdentity.userName AS user
EVAL-action = case('responseElements.ConsoleLogin'="Success","success",
                  'responseElements.ConsoleLogin'="Failure","failure",
                  isNotNull(errorMessage),"failure", true(),"success")

[WinEventLog:Security]
TRANSFORMS-route_identity = route_identity
FIELDALIAS-norm_src = Source_Network_Address AS src
EVAL-user = mvindex(Account_Name, -1)

[access_combined]
FIELDALIAS-norm_src = clientip AS src
EVAL-action = case(status>=200 AND status<400,"success",
                  status>=400,"failure", true(),"unknown")

```

transforms.conf

```

[route_identity]
REGEX    = EventCode=(4624|4625|4634|4648|4672|4720|4722|4724|4726|4728|4732|4738|4740|4768|
4769|4771|4776)
DEST_KEY = _MetaData:Index
FORMAT   = identity

```

Service configuration

```

sc.exe config splunkd obj= LocalSystem
& "C:\Program Files\Splunk\bin\splunk.exe" restart

```

Appendix B: Validation Queries (SPL)

Source health check

```
| tstats count latest(_time) as last_event where index=* by index, sourcetype
| eval last_event=strftime(last_event,"%Y-%m-%d %H:%M:%S")
| sort index
```

Failed authentication across three sources, with normalized fields

```
(index=win sourcetype="WinEventLog:Security" EventCode=4625)
OR (index=cloud eventName=ConsoleLogin action=failure)
OR (index=app uri_path="/login" action=failure)
| eval source_type=case(index="win","Windows", index="cloud","AWS CloudTrail",
index="app","Web App")
| stats count by source_type, src, user
```

Cloud persistence and defense-evasion sequence

```
index=cloud eventName IN (CreateUser, AttachUserPolicy, CreateAccessKey, StopLogging)
| sort _time
| table _time eventName "userIdentity.userName" sourceIPAddress
```

Bulk object download (exfiltration)

```
index=cloud eventName=GetObject
| stats count min(_time) as first_seen max(_time) as last_seen
  by "userIdentity.userName", sourceIPAddress, "requestParameters.bucketName"
| eval duration_sec=last_seen-first_seen
| fieldformat first_seen=strftime(first_seen,"%Y-%m-%d %H:%M:%S")
| fieldformat last_seen=strftime(last_seen,"%Y-%m-%d %H:%M:%S")
```

Account lifecycle on Windows

```
index=identity EventCode IN (4720,4732,4726)
| sort _time
| table _time EventCode Account_Name
```

Appendix C: Simulated Data Composition

Both simulated sources mix benign activity with attack behavior, so that Week 2 detections can be tested for both detection and false positives.

Table 12. *Composition of the simulated datasets*

Dataset	Events	Composition
Web access log	220	150 benign requests from internal clients; 40 scanner requests returning 404; 4 SQL injection requests returning 500; 25 failed and 1 successful login on <code>/login</code>
CloudTrail	102	32 benign events (12 console logins, 10 <code>ListBuckets</code> , 10 <code>DescribeInstances</code>); 15 failed and 1 successful console login; 3 persistence events; 1 <code>StopLogging</code> ; 50 <code>GetObject</code>

The generator scripts write the files to `C:\SOCLab\logs\`. Timestamps are produced relative to the time of generation so that events fall inside Splunk's default search windows.

Week 1 Deliverables Verification

Final check of the five Week 1 deliverables required by the project booklet (Project 4: SIEM Detection Engineering, Alert Triage & SOC Operations). Page numbers refer to this document.

Table 13. *Final deliverables checklist with page references*

#	Required deliverable	Evidence in this report	Location	Status
1	SIEM architecture diagram	Architecture diagram, layer descriptions and index design	Section 3, pp. 4–6 Figure 1 on p. 6	Verified
2	Data source inventory	Inventory of 7 sources with category, collection method, index, sourcetype and live or simulated status	Section 4, p. 7	Verified
3	Log onboarding evidence	Configuration, validation searches and screenshots for all five telemetry categories; source health check	Section 5, pp. 8–15 Section 7, pp. 18–20	Verified
4	Normalized field mapping	CIM-aligned mapping table and before/after screenshots for Windows, CloudTrail and web logs	Section 6, pp. 16–17	Verified
5	Retention plan	Per-index retention policy and indexes.conf snippet	Section 5.8, p. 15	Documented (proposed)

Verification notes

- **Verified** means the evidence (configuration, query result or screenshot taken from the lab) is included in this report.
- **Retention plan:** a proposed policy, not enforced in the lab. The lab runs with Splunk's default retention.
- **Normalization:** CIM-aligned field naming only. The CIM add-on and data models were not installed.
- **Simulated sources:** the web access log and AWS CloudTrail events are generated with the real log schemas, because no production web server or cloud account was available (Appendix C).
- **The linux index** is created but empty; it is reserved for a Linux server in later weeks.